Switch AI

ai.switchsupply.com

Pre-employment Exam

Senior Platform Architect

Duration	3-5 days
Format	Take-home coding task
Submission	GitHub repo + Loom/video walkthrough
Stack required	Next.js + Supabase + TypeScript
AI required	OpenAI API or any other free model (use free trial)
Deployment	Vercel (free tier)

Context — What you are building

We are building an internal supply chain SaaS platform for an Australian ingredient supplier. One of our core features is an **AI-powered query assistant** that lets the sales team ask natural language questions about product data and get back data-backed answers sourced from our database.

For this exam, you will build a **simplified version of this feature** — a mini product intelligence dashboard with an AI query layer. This is not a toy project. We want to see how you would actually approach building this for a real production environment.

Required stack

- Next.js 14+ (App Router)
- TypeScript
- Supabase (PostgreSQL + Auth)
- UI component libraries: Tailwind CSS & ShadCN framework
- OpenAI API (gpt-4o or gpt-4o-mini)
- Prisma or Supabase client
- Vercel deployment

The tasks

1. Database schema + seed data (20 pts)

Design and create a Supabase PostgreSQL schema for a simple ingredient/product catalogue. The schema must include:

- A products table — name, SKU, category, unit of measure, unit price, stock quantity, supplier name, country of origin
- A price_history table — product ID, price, date recorded, currency
- Row-level security (RLS) policies — only authenticated users can read data
- Seed the database with at least 15 realistic product records (food ingredients — e.g. matcha powder, beetroot powder, spirulina, etc.) with at least 3 months of price history per product

Include your migration SQL file in the repo. We will run it ourselves to verify the schema.

2. Product dashboard page (25 pts)

Build a clean, functional product dashboard page at `/dashboard` that displays:

- A searchable, filterable product table — filter by category, sort by price or stock
- A product detail panel — clicking a row shows the product details including price history as a simple chart or table
- A summary header — total products, total categories, average unit price
- Authentication gate — redirect to login if the user is not authenticated (use Supabase Auth)
- A simple login page at `/login` — email + password only

We are not looking for a beautiful UI. We are looking for clean, readable component structure, good TypeScript typing, and sensible use of Next.js App Router patterns (server components, client components, loading states).

3. AI query assistant API route (30 pts)

This is the most important task. Build a `POST /api/ai/query` route that accepts a natural language question and returns a structured AI-powered answer sourced from the database.

Requirements:

- Accept a `{ question: string }` body
- Fetch the relevant data from Supabase based on the question context
- Send the question + relevant data to OpenAI and return a structured response
- The response must include: the answer, the source data used (product names, SKUs, prices), and a confidence indicator (high / medium / low)
- Handle edge cases: no relevant data found, vague questions, database errors

Example questions the endpoint must handle correctly:

- *"What is the current price of matcha powder?"*
- *"Which ingredients are running low on stock?"*
- *"Show me the price trend for spirulina over the last 3 months."*
- *"Which supplier has the most products in our catalogue?"*

The AI must reference specific product records — not hallucinate. Include the source data in the response so the answer can be traced back to the database.

4. AI chat UI on the dashboard (15 pts)

Add a simple AI query panel to the dashboard page that calls the `/api/ai/query` endpoint and displays the response. Requirements:

- A text input where the user can type a question and submit
- Display the AI answer clearly
- Show the source data references (which products / records the answer was based on)
- Show the confidence indicator badge — green (high), amber (medium), red (low)
- Loading state while the API call is in progress
- Error state if the query fails

5. Architecture decision record (ADR) 10 pts

Include a `ARCHITECTURE.md` file in your repo that documents the following in plain English (no more than 1 page):

- How you structured the project and why
- One architectural decision you made that you'd do differently if this were a production multi-tenant SaaS with 1,000+ tenants
- What would break first if this app had to handle 10,000 AI queries per day — and how would you fix it

This is not a test of your writing. It is a test of your architectural thinking. Short and direct is better than long and vague.

Bonus tasks (optional — for extra credit)

- +5 pts Streaming AI response — stream the OpenAI response token by token instead of waiting for the full reply
- +5 pts Query history — store each AI question and answer in a `query_history` Supabase table and show the last 5 queries on the dashboard
- +5 pts Rate limiting — limit the AI query endpoint to 20 requests per user per hour using a simple counter in Supabase or Redis

Scoring breakdown

Task	Title	Points
1	Database schema + seed data	20 points
2	Product dashboard page	25 points
3	AI query assistant API route	30 points
4	AI chat UI	15 points
5	Architecture decision record	10 points
	Total	**100 points**

Bonus tasks (optional) +15 pts

Submission requirements

- **GitHub repo** — public or shared with us. Include a clear README with setup instructions, environment variables needed, and how to run locally
- **Live Vercel URL** — deployed and accessible. We will test it ourselves
- **Loom or screen recording (5–10 mins)** — walk us through your code and explain your key decisions. This replaces the first interview screen
- **SQL migration file** — included in the repo so we can recreate your schema
- **No AI-generated boilerplate without explanation** — if you used Cursor or Claude Code to help, mention it in your README. We encourage the use of AI tools — we just want to know what you built vs. what the AI built

What we are evaluating

We are NOT looking for a pixel-perfect UI. We ARE looking for: clean code structure, good TypeScript typing, sensible Next.js App Router patterns, a working AI endpoint that actually traces answers back to the database, and an architecture document that shows how you think — not just what you can copy.